

華中科技大學

课程实验报告

课程名称： 深度学习

实验名称： 实验三 VAE

院 系： 计算机科学与技术学院

专业班级： 本硕博 2301 班

学 号： U202315763

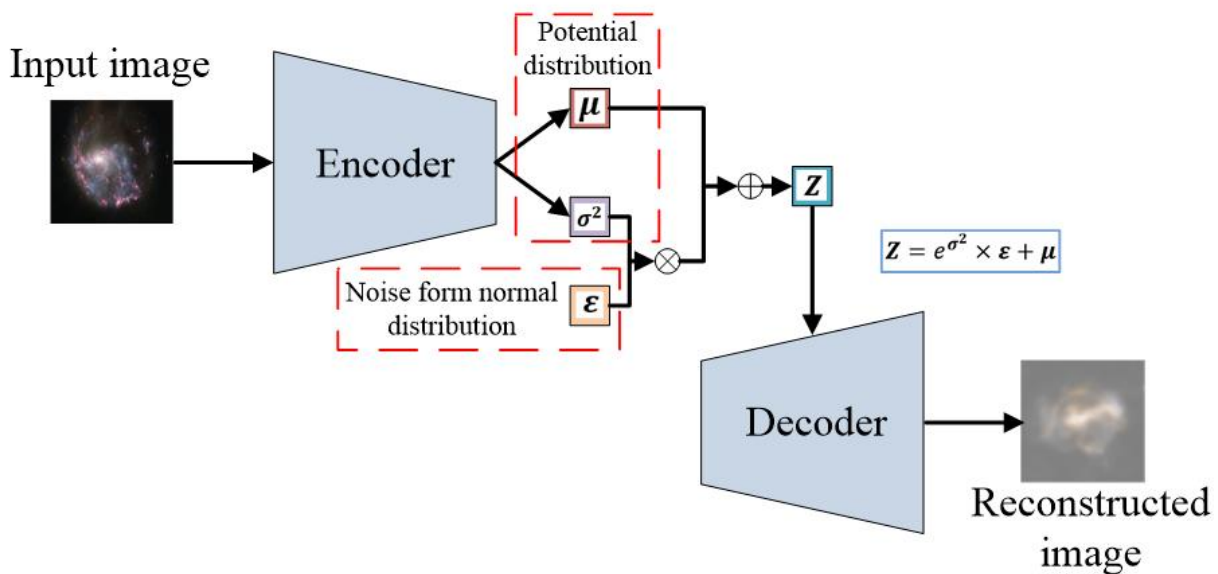
姓 名： 王家乐

2025 年 12 月 30 日

1. 实验背景与目的

1.1 变分自编码器（Variational Autoencoder）

变分自编码器（VAE）是一种深度生成模型，它结合了自编码器和变分推断的思想，能够通过学习潜在空间的分布来生成与输入数据相似的全新数据。VAE模型的核心在于其重参数化技巧，它通过引入潜在变量的概率分布来进行推断，而不仅仅是通过确定性编码器生成隐含表示，从而能够进行更有效的生成任务。VAE 广泛应用于图像生成、数据生成、降维、异常检测等领域。与传统自编码器相比，VAE 的生成能力更强，能够产生高质量的样本，尤其在无监督学习和生成对抗网络（GANs）等领域中显示出了巨大的潜力。VAE 结构图如下：



1.2 实验目的

本实验的目的在于通过实现变分自编码器（VAE）模型，深入理解深度生成模型的基本原理及其在图像生成任务中的应用。实验分为基本任务和进阶任务两部分，主要要求在 MNIST 手写数字数据集上完成模型的构建、训练与评估。基

本任务涉及到从 MNIST 数据集中加载数字 0 的图像并将其格式化为适合 VAE 模型输入的格式，构建 VAE 的编码器、重参数化模块和解码器，设定合适的训练超参数并进行训练。在测试集上评估 VAE 的重构能力，并通过从潜在空间进行随机采样生成新的数字 0 图像。进阶任务则要求实现条件变分自编码器（Conditional VAE），在该模型中加入条件信息，使得模型能够生成特定数字标签对应的图像。旨在加深对 VAE 模型的理解，并掌握其在图像生成中的应用。

2. 实验设计

2.1 数据加载与预处理

在基本任务中，需要从 MNIST 数据集加载所有数字 0 的图像，并进行适当的预处理以适应 VAE 模型的输入需求。首先，使用 `torchvision.datasets.MNIST` 来加载 MNIST 数据集，并利用 `transforms.ToTensor()` 将图片转换为 Tensor 格式并进行归一化。接着，筛选出标签为 0 的图像，并通过 `DataLoader` 将数据加载。由于 VAE 模型是基于图像的重构，因此只选取数字 0 的图像进行训练和评估。

在进阶任务中，数据加载与预处理的过程与基本任务类似，但需要进一步对数据进行条件化处理。在此任务中，除了加载数字图像外，还需要对标签进行 `onehot` 编码处理，将标签作为条件信息与图像一同输入编码器。在训练过程中，条件信息（即数字标签）将与图像拼接，作为编码器的输入。

2.2 基本任务 VAE 模型结构

VAE 模型的结构由三个主要部分组成：编码器（Encoder）、重参数化（Reparameterize）模块和解码器（Decoder）。编码器的主要功能是将输入图像

映射到潜在空间的均值和对数方差（log variance）；重参数化模块则保证了从潜在空间中采样的过程可微，允许进行反向传播和梯度更新；解码器则负责从潜在空间样本重建图像。

• 编码器

编码器部分使用卷积神经网络（CNN）来提取图像特征。输入的图片首先通过两个卷积层（Conv2d），逐步从 28×28 的图片尺寸减小至 7×7 的特征图。每个卷积层后都跟着 ReLU 激活函数，用于增加模型的非线性。接着，经过 Flatten 层将特征图展开为一维向量，再通过两个全连接层分别输出潜在空间的均值（mu）和对数方差（logvar）。这些值将用于后续的重参数化步骤。

```
class Encoder(nn.Module):
    def __init__(self, latent_dim: int):
        super().__init__()
        self.net = nn.Sequential(
            # 卷积层 1: 输入 1 通道(灰度图), 输出 32 通道, 尺寸减半 (28x28 -> 14x14)
            nn.Conv2d(1, 32, kernel_size=4, stride=2, padding=1),
            nn.ReLU(inplace=True),
            # 卷积层 2: 输出 64 通道, 尺寸再次减半 (14x14 -> 7x7)
            nn.Conv2d(32, 64, kernel_size=4, stride=2, padding=1),
            nn.ReLU(inplace=True),
            nn.Flatten(), # 展平: 64 * 7 * 7
        )
        # 两个全连接层分别预测均值(mu)和对数方差(logvar)
        self.fc_mu = nn.Linear(in_features=64 * 7 * 7, out_features=latent_dim)
        self.fc_logvar = nn.Linear(in_features=64 * 7 * 7, out_features=latent_dim)

    def forward(self, x):
        h = self.net(x)
        mu = self.fc_mu(h)
        logvar = self.fc_logvar(h)
        return mu, logvar
```

• 重参数化模块

在 VAE 的训练过程中，潜在空间的采样需要通过重参数化来实现。reparameterize 方法通过均值和对数方差计算潜在变量的标准差（std），然后从

标准正态分布中采样一个噪声向量（`eps`），最终生成潜在变量 z 。这个过程使得潜在空间的采样变得可微，从而允许在训练中进行梯度更新。

```
class VAE(nn.Module):
    .....
    @staticmethod
    def reparameterize(mu, logvar):
        # 重参数化:  $z = \mu + \sigma * \epsilon$ 
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + std * eps
    .....
```

• 解码器

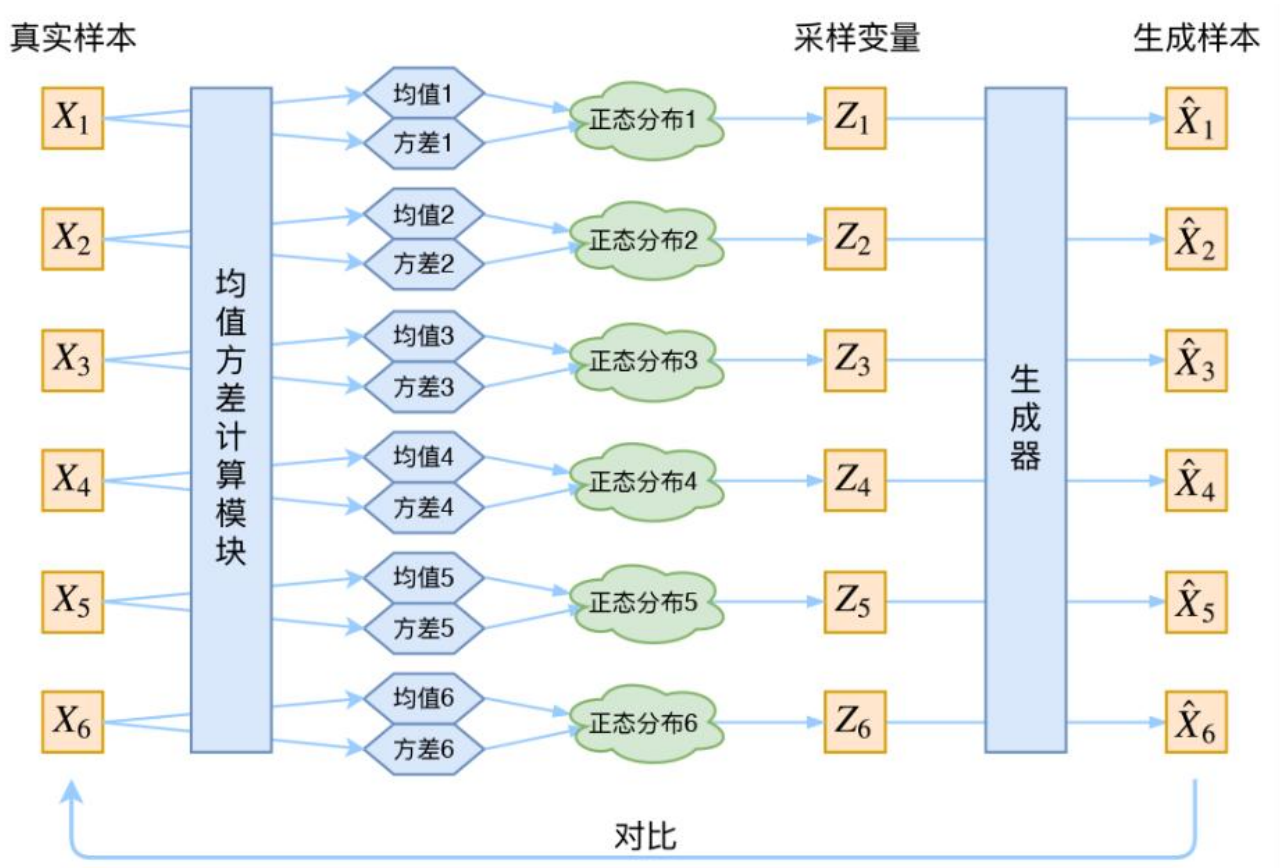
解码器的作用是将潜在空间中的样本（即从潜在变量中采样的 z ）重建回原始图像。解码器首先通过一个全连接层将潜在向量 z 转化为形状为 $64 \times 7 \times 7$ 的张量，然后使用反卷积（`ConvTranspose2d`）逐步恢复图像的空间结构。经过两次反卷积操作，图像的尺寸从 7×7 恢复到 28×28 。最后，解码器输出的是图像的 logits，这些 logits 会在损失函数中与输入图像的真实值进行比较。

```
class Decoder(nn.Module):
    def __init__(self, latent_dim: int):
        super().__init__()
        # 将隐变量映射回特征图所需的维度
        self.fc = nn.Linear(in_features=latent_dim, out_features=64 * 7 * 7)
        self.net = nn.Sequential(
            # 反卷积1: 上采样 (7x7 -> 14x14)
            nn.ConvTranspose2d(64, 32, kernel_size=4, stride=2, padding=1),
            nn.ReLU(inplace=True),
            # 反卷积2: 上采样 (14x14 -> 28x28), 恢复到原图尺寸
            nn.ConvTranspose2d(32, 1, kernel_size=4, stride=2, padding=1),
        )

    def forward(self, z):
        # 先通过全连接层, 再 reshape 成特征图形状 (B, 64, 7, 7)
        h = self.fc(z).view(z.size(0), 64, 7, 7)
        logits = self.net(h)
        return logits
```

整个 VAE 模型的结构通过 VAE 类实现，它将编码器、解码器和重参数化模

块组合在一起。在 `forward` 方法中，输入图像通过编码器得到潜在空间的均值和对数方差，之后通过重参数化采样潜在变量 z ，再通过解码器生成图像的 logits。最终的输出包括重构的图像 logits、潜在空间的均值 μ 和对数方差 $\log\sigma^2$ ，这些将用于后续的损失计算。



2.3 进阶任务 Conditional VAE 模型结构

在进阶任务中，Conditional VAE 相较于基本任务中的 VAE 模型进行了关键的修改，以引入条件信息（即标签）来生成特定类别的图像。Conditional VAE 将条件信息与图像输入和潜在空间的样本一同处理，从而使得模型不仅能够学习图像的潜在表示，还能根据给定的条件生成指定类别的图像。

首先，编码器部分发生了变化。在基本 VAE 中，编码器仅接受图像作为输入，将其映射到潜在空间的均值和对数方差。而在 Conditional VAE 中，编码器的输

入不仅包括图像数据，还包括条件信息。具体来说，条件信息表示为一个 onehot 编码向量，这个向量扩展后与输入图像拼接一起输入到卷积网络中。

```
class Encoder(nn.Module):
    def __init__(self, latent_dim: int, cond_dim: int):
        super().__init__()
        self.cond_dim = cond_dim
        # 输入通道数 = 图像通道(1) + 条件维度(cond_dim)
        self.net = nn.Sequential(
            nn.Conv2d(1 + cond_dim, 32, kernel_size=4, stride=2, padding=1), # 28x28 -> 14x14
            nn.ReLU(inplace=True),
            nn.Conv2d(32, 64, kernel_size=4, stride=2, padding=1),          # 14x14 -> 7x7
            nn.ReLU(inplace=True),
            nn.Flatten(),
        )
        self.fc_mu = nn.Linear(64 * 7 * 7, latent_dim)
        self.fc_logvar = nn.Linear(64 * 7 * 7, latent_dim)

    def forward(self, x, y_onehot):
        # 将条件向量扩展为特征图大小: (B, cond_dim) -> (B, cond_dim, 28, 28)
        y_map = y_onehot[:, :, None, None].expand(-1, -1, x.size(2), x.size(3))
        # 在通道维度拼接图像和条件图: (B, 1+cond_dim, 28, 28)
        x_in = torch.cat([x, y_map], dim=1)
        h = self.net(x_in)
        return self.fc_mu(h), self.fc_logvar(h)
```

其次，解码器也进行了类似的修改。在基本 VAE 的解码器中，潜在变量被输入到全连接层，并通过反卷积层逐步恢复图像。而在 Conditional VAE 中，解码器的输入不仅包含潜在变量，还需要加入条件信息。具体来说，条件信息与潜在变量一起被拼接，作为解码器的输入。

```
class Decoder(nn.Module):
    def __init__(self, latent_dim: int, cond_dim: int):
        super().__init__()
        # 输入维度: 隐变量 z + 条件向量 y
        self.fc = nn.Linear(latent_dim + cond_dim, 64 * 7 * 7)
        self.net = nn.Sequential(
            nn.ConvTranspose2d(64, 32, kernel_size=4, stride=2, padding=1), # 7x7 -> 14x14
            nn.ReLU(inplace=True),
            nn.ConvTranspose2d(32, 1, kernel_size=4, stride=2, padding=1),  # 14x14 -> 28x28
        )
```



```
def forward(self, z, y_onehot):  
    # 拼接 z 和 y: (B, latent_dim + cond_dim)  
    zy = torch.cat([z, y_onehot], dim=1)  
    # 映射回特征图形状: (B, 64, 7, 7)  
    h = self.fc(zy).view(z.size(0), 64, 7, 7)  
    return self.net(h)
```

其余模块保持不变，但由于在编码器和解码器中都引入了条件信息，因此每个步骤中都需要确保条件信息能够与图像数据一同处理。在 forward 方法中，输入的标签 y 会转换为独热编码，并与输入图像一同传递给编码器和解码器。这使得模型能够基于输入的条件生成与该条件相对应的图像。

2.4 损失函数

VAE 的损失函数由两部分组成：重构损失（reconstruction loss）和 KL 散度损失（KL divergence），并且使用了一个调节因子 beta 来控制 KL 散度的权重。

重构损失计算的是模型生成的图像与原始输入图像之间的差异。这里使用了 BCEWithLogitsLoss，它结合了 Sigmoid 激活和二元交叉熵损失函数。x_logits 是解码器输出的未经过 Sigmoid 激活的 logits，x 是原始图像数据。首先计算每个像素的损失，接着通过将每个样本的像素损失加总再通过求得整个批次的平均损失。因此，重构损失反映了所有图像的重建质量。

KL 散度损失度量了编码器输出的潜在空间分布（通过均值 mu 和对数方差 logvar 表示）与标准正态分布之间的差异。KL 散度是变分推断中的重要组成部分，确保潜在空间的分布接近标准正态分布。公式 $-0.5 * \text{torch.sum}(1 + \logvar - \mu.\text{pow}(2) - \logvar.\text{exp}(), \text{dim}=1)$ 计算了每个样本的 KL 散度，最后得到整个批次的 KL 散度平均值。

最终的总损失是由重构损失和 KL 散度损失的加权和构成，其中 beta 是一个超参数，用于调节 KL 散度在总损失中的权重。beta 的值越大，模型越倾向于优

化潜在空间的结构，可能会导致重构质量下降；而 β 较小时，模型会更加关注重构图像的质量，从而可能牺牲潜在空间的结构。VAE 的损失函数不仅考虑了生成图像的质量，还引入了对潜在空间结构的约束，确保了生成模型能够有效地学习到潜在空间的有意义的分布。

$$\text{loss} = \text{MSE}(X, X') + \text{KL}(N(\mu_1, \sigma_1^2), N(0, 1))$$

```
def vae_losses(x, x_logits, mu, logvar, beta: float = 1.0):
    # recon: sum over pixels, mean over batch
    recon = F.binary_cross_entropy_with_logits(x_logits, x, reduction='none')
    recon = recon.flatten(1).sum(dim=1).mean()
    # KL: mean over batch
    kl_per_sample = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp(), dim=1)
    kl = kl_per_sample.mean()

    loss = recon + beta * kl
    return loss, recon, kl
```

2.5 训练与评估

在 VAE 的训练与评估部分，相关超参数由 CFG 类配置。优化器使用 Adam，训练过程中的损失、重构损失、KL 散度损失以及测试集上的重构损失都被记录在 history 字典中，以便后续分析和可视化。

```
@dataclass
class CFG:
    data_root: str = "./data"
    batch_size: int = 128
    lr: float = 1e-3
    epochs: int = 20
    latent_dim: int = 16 # 隐变量维度
    beta: float = 1.0 # KL 最终权重 (=1 即标准 VAE)
    beta_warmup_epochs: int = 5 # 前 N 个 epoch 线性 warm-up 到 beta
    num_workers: int = 2
    pin_memory: bool = True
```

训练过程的核心函数是 `run_epoch_train`，它在每个 epoch 中执行一次模型的训练。模型通过前向传播得到重构图像 `x_logits` 以及潜在空间的均值 (`mu`) 和对

数方差（logvar）。接着使用自定义的 `vae_losses` 函数计算损失，包括重构损失和 KL 散度损失，并根据 `beta` 的值加权求和得到总损失。损失反向传播并进行优化器更新。`get_beta(epoch)`函数控制在训练初期 `beta` 的线性 warm-up 过程，使得 KL 散度损失在训练初期逐渐增加，避免过早地对潜在空间的约束。

```
def get_beta(epoch: int) -> float:
    # 线性 warmup: 让 KL 在训练早期权重较小, 避免模型过早陷入后验坍塌
    if cfg.beta_warmup_epochs <= 0:
        return cfg.beta
    t = min(1.0, epoch / cfg.beta_warmup_epochs)
    return cfg.beta * t

def run_epoch_train(model, loader, epoch: int):
    model.train()
    tot_loss = 0.0
    tot_recon = 0.0
    tot_kl = 0.0
    n = 0
    beta = get_beta(epoch) # 获取当前 epoch 对应的 beta 值
    for x, _ in loader:
        x = x.to(device)
        optimizer.zero_grad(set_to_none=True)
        x_logits, mu, logvar = model(x)
        # 计算损失 (Total = Recon + beta * KL)
        loss, recon, kl = vae_losses(x, x_logits, mu, logvar, beta=beta)
        # 反向传播与参数更新
        loss.backward()
        optimizer.step()
        # 累积统计量
        bs = x.size(0)
        tot_loss += loss.item() * bs
        tot_recon += recon.item() * bs
        tot_kl += kl.item() * bs
        n += bs
    return tot_loss / n, tot_recon / n, tot_kl / n, beta
```

评估过程与训练类似，测试集上的数据通过 `test_loader` 加载，并送入设备进行前向传播，计算重构损失并进行累加。评估过程中不涉及反向传播，因此更轻量化。每个 `epoch` 结束时，测试集上的平均重构损失被记录下来。

```
@torch.no_grad()
```

```
def run_epoch_test(model, loader, epoch: int):
    model.eval()
    tot_recon = 0.0
    n = 0
    beta = get_beta(epoch)
    for x, _ in loader:
        x = x.to(device)
        x_logits, mu, logvar = model(x)
        # 验证阶段主要关注重构误差
        _, recon, _ = vae_losses(x, x_logits, mu, logvar, beta=beta)
        bs = x.size(0)
        tot_recon += recon.item() * bs
        n += bs
    return tot_recon / n
```

整个训练过程通过循环遍历所有 epoch。在每个 epoch 中，训练集和测试集的损失都会被打印出来，并记录到 history 中。beta 的值在每个 epoch 中动态调整，随训练进展逐步增大，确保模型逐渐学习到潜在空间的结构，同时保持图像重构的质量。训练和评估的过程不断迭代，直到达到设定的训练轮数。

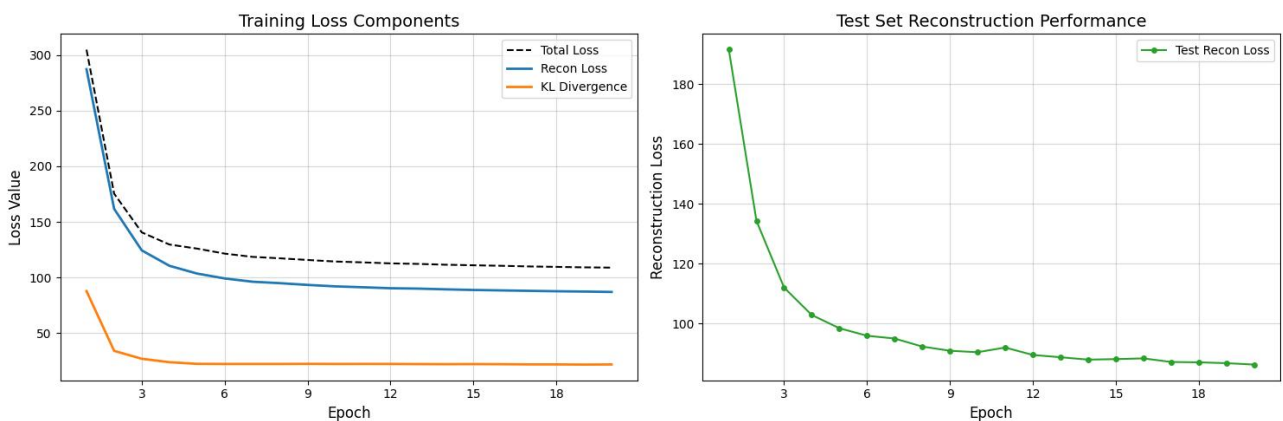
3. 实验结果

3.1 基本任务

基本任务的超参数为 learning_rate=0.001，batch_size=128，epoch=20。训练过程中，记录训练集在模型上的总损失、重构损失、KL 散度损失，测试集在模型上的 KL 散度损失，训练 log 如下及曲线如下：

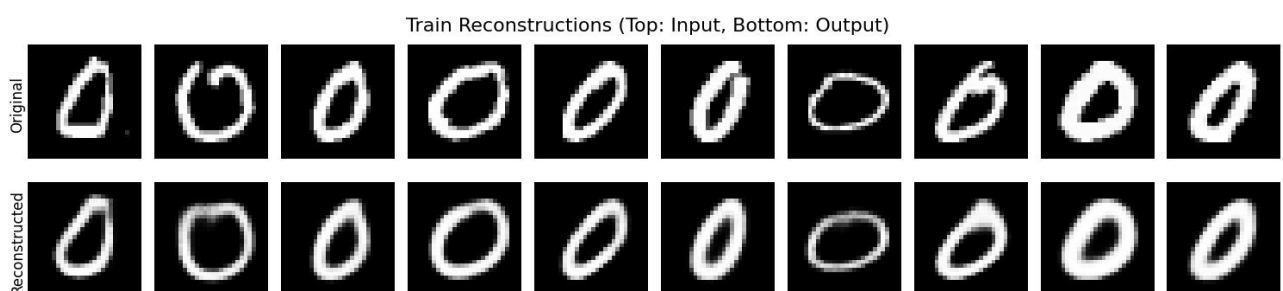
Epoch 01/20	beta=0.20	train_total=304.85	recon=287.28	kl=87.84	test_recon=191.49	
Epoch 02/20	beta=0.40	train_total=175.39	recon=161.75	kl=34.10	test_recon=134.28	
Epoch 03/20	beta=0.60	train_total=140.60	recon=124.41	kl=26.98	test_recon=111.96	
Epoch 04/20	beta=0.80	train_total=129.69	recon=110.56	kl=23.91	test_recon=102.81	
Epoch 05/20	beta=1.00	train_total=126.02	recon=103.59	kl=22.43	test_recon= 98.38	
Epoch 06/20	beta=1.00	train_total=121.52	recon= 99.19	kl=22.33	test_recon= 95.87	
Epoch 07/20	beta=1.00	train_total=118.61	recon= 96.26	kl=22.34	test_recon= 94.95	
Epoch 08/20	beta=1.00	train_total=117.29	recon= 94.95	kl=22.34	test_recon= 92.26	
Epoch 09/20	beta=1.00	train_total=115.80	recon= 93.38	kl=22.42	test_recon= 90.86	
Epoch 10/20	beta=1.00	train_total=114.41	recon= 92.08	kl=22.34	test_recon= 90.37	

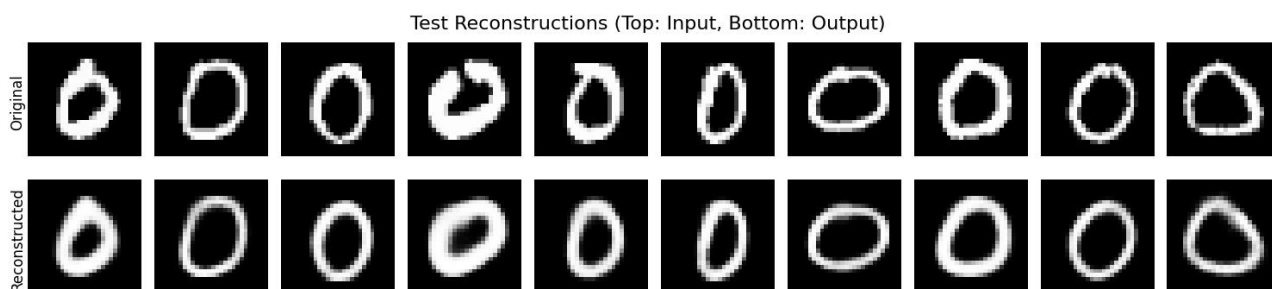
Epoch 11/20	beta=1.00	train_total=113.64	recon= 91.27	kl=22.36	test_recon= 91.94
Epoch 12/20	beta=1.00	train_total=112.74	recon= 90.39	kl=22.35	test_recon= 89.45
Epoch 13/20	beta=1.00	train_total=112.26	recon= 90.06	kl=22.21	test_recon= 88.69
Epoch 14/20	beta=1.00	train_total=111.53	recon= 89.40	kl=22.13	test_recon= 87.89
Epoch 15/20	beta=1.00	train_total=111.03	recon= 88.83	kl=22.20	test_recon= 88.06
Epoch 16/20	beta=1.00	train_total=110.57	recon= 88.46	kl=22.11	test_recon= 88.31
Epoch 17/20	beta=1.00	train_total=110.00	recon= 88.07	kl=21.93	test_recon= 87.11
Epoch 18/20	beta=1.00	train_total=109.60	recon= 87.69	kl=21.91	test_recon= 86.99
Epoch 19/20	beta=1.00	train_total=109.18	recon= 87.41	kl=21.77	test_recon= 86.71
Epoch 20/20	beta=1.00	train_total=108.93	recon= 87.06	kl=21.87	test_recon= 86.24



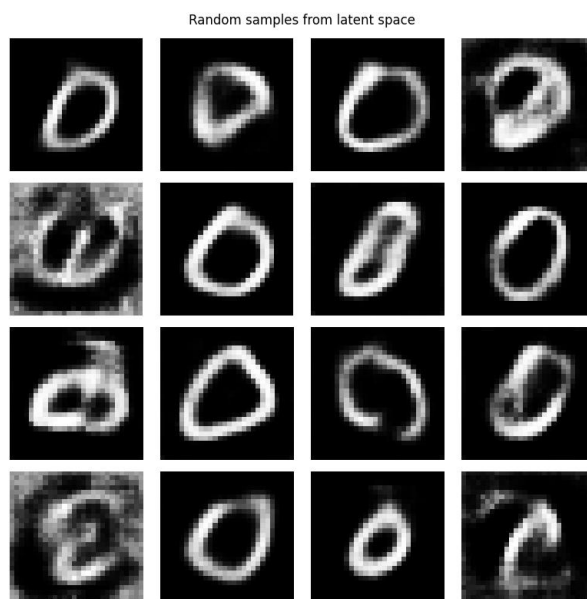
可见，在训练初期，VAE 模型的总损失、重构损失和 KL 散度损失迅速下降，尤其是在前几个 epoch 内，说明模型正在逐步学习如何有效重建输入图像，同时逐渐优化潜在空间的分布。KL 散度损失的快速下降表明在训练初期对潜在空间的约束逐步加强，而重构损失的稳定下降则表明模型逐渐提高了图像重建的质量。随着训练的进行，模型的总损失趋于平稳，且测试集上的重构损失也逐渐收敛，表明模型的生成能力已接近稳定并具有较好的泛化能力。

接下来分别从训练集和测试集中取部分样本，输入到 VAE 中获取输出，并与原图比较，如下两图所示：





最后在潜空间随机采样噪声样本，并进行生成，展示生成结果如下图所示：

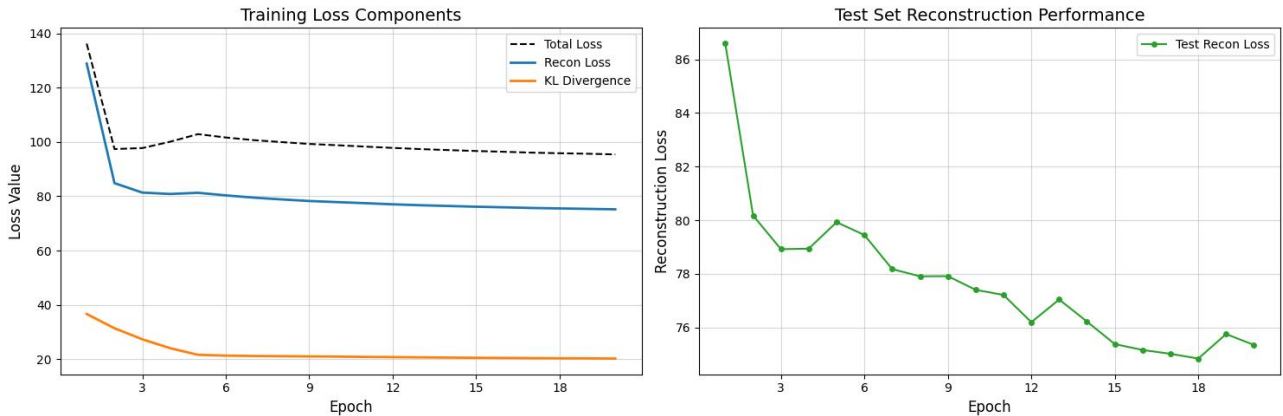


3.2 进阶任务

进阶任务的超参数为 `learning_rate=0.001`，`batch_size=128`，`epoch=20`。训练过程中，仍然记录训练集在模型上的总损失、重构损失、KL 散度损失，测试集在模型上的 KL 散度损失，训练 log 如下及曲线如下：

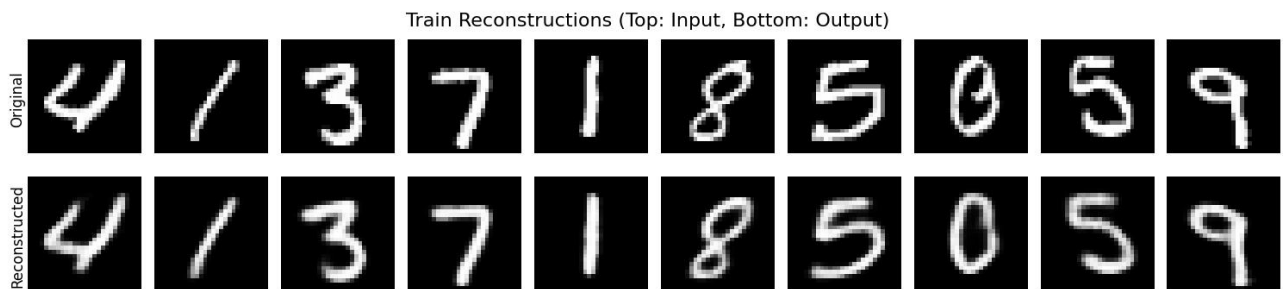
Epoch 01/20		beta=0.20		train_total=136.17	recon=128.85	kl=36.63		test_recon= 86.59	
Epoch 02/20		beta=0.40		train_total= 97.37	recon= 84.81	kl=31.39		test_recon= 80.18	
Epoch 03/20		beta=0.60		train_total= 97.73	recon= 81.34	kl=27.31		test_recon= 78.92	
Epoch 04/20		beta=0.80		train_total=100.07	recon= 80.83	kl=24.05		test_recon= 78.94	
Epoch 05/20		beta=1.00		train_total=102.86	recon= 81.26	kl=21.60		test_recon= 79.93	
Epoch 06/20		beta=1.00		train_total=101.62	recon= 80.31	kl=21.31		test_recon= 79.45	
Epoch 07/20		beta=1.00		train_total=100.66	recon= 79.48	kl=21.17		test_recon= 78.18	
Epoch 08/20		beta=1.00		train_total= 99.93	recon= 78.83	kl=21.10		test_recon= 77.91	
Epoch 09/20		beta=1.00		train_total= 99.25	recon= 78.24	kl=21.01		test_recon= 77.92	
Epoch 10/20		beta=1.00		train_total= 98.77	recon= 77.83	kl=20.94		test_recon= 77.41	
Epoch 11/20		beta=1.00		train_total= 98.27	recon= 77.44	kl=20.83		test_recon= 77.22	

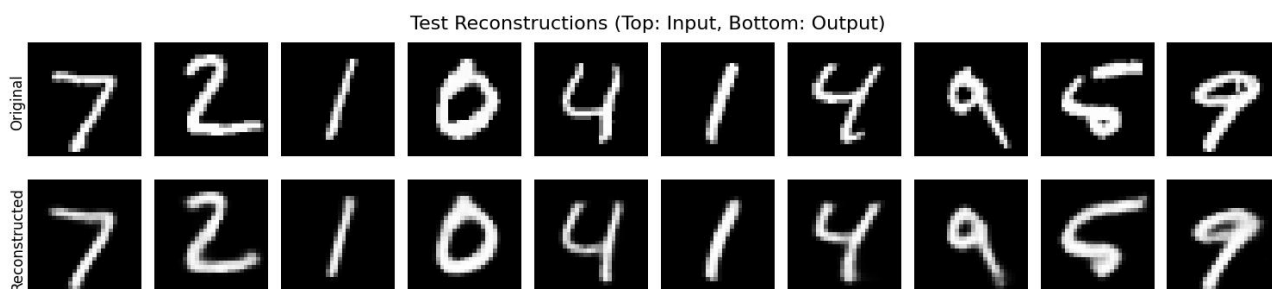
Epoch 12/20		beta=1.00		train_total= 97.78	recon= 77.02	kl=20.77		test_recon= 76.20	
Epoch 13/20		beta=1.00		train_total= 97.35	recon= 76.68	kl=20.67		test_recon= 77.05	
Epoch 14/20		beta=1.00		train_total= 96.99	recon= 76.40	kl=20.59		test_recon= 76.22	
Epoch 15/20		beta=1.00		train_total= 96.65	recon= 76.14	kl=20.51		test_recon= 75.38	
Epoch 16/20		beta=1.00		train_total= 96.37	recon= 75.91	kl=20.46		test_recon= 75.16	
Epoch 17/20		beta=1.00		train_total= 96.05	recon= 75.67	kl=20.37		test_recon= 75.02	
Epoch 18/20		beta=1.00		train_total= 95.83	recon= 75.51	kl=20.32		test_recon= 74.84	
Epoch 19/20		beta=1.00		train_total= 95.64	recon= 75.35	kl=20.30		test_recon= 75.76	
Epoch 20/20		beta=1.00		train_total= 95.39	recon= 75.17	kl=20.22		test_recon= 75.35	



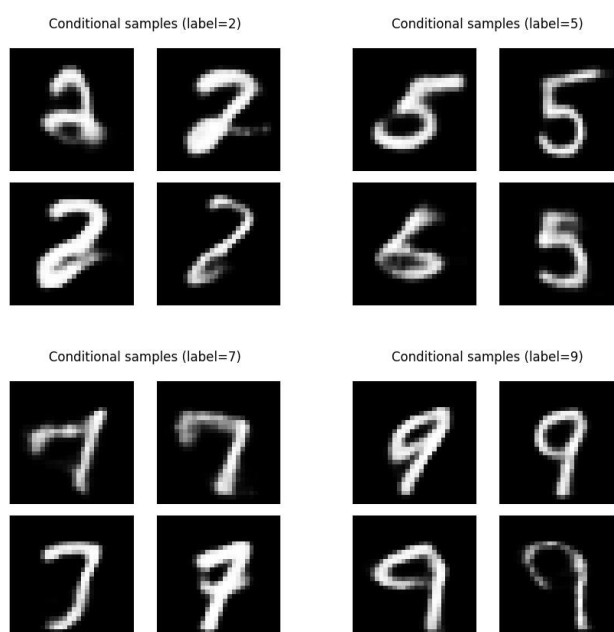
Conditional VAE 的训练过程在前几个 epoch 内表现出明显的下降趋势，尤随后趋于平稳，表明模型逐渐学会了根据条件生成图像，以及潜在空间的分布逐渐收敛到期望的标准正态分布。测试集上的重构损失也表现出类似的下降趋势。整体来看，Conditional VAE 在训练过程中逐步收敛，表明模型能够有效地根据条件信息生成高质量的图像，且具备较好的泛化能力。

接下来分别从训练集和测试集中取部分样本，输入到 VAE 中获取输出，并与原图比较，如下两图所示：





最后在潜空间随机采样噪声样本，测试条件信息（标签）分别为 2、5、7、9 时模型的生成效果，展示生成结果如下图所示：



4. 总结与体会

本次变分自编码器（VAE）实验中，通过构建并训练 VAE 模型，深刻理解了其在图像生成领域的应用，尤其是在处理 MNIST 数据集中的数字图像时，模型能够通过学习潜在空间的分布来有效地重构输入图像并生成新的图像。实验中，我发现 VAE 的训练初期，重构损失和 KL 散度损失下降较快，标志着模型在逐步学习如何生成高质量的图像，同时优化潜在空间的结构。随着训练的进行，损失逐渐趋于平稳，表明模型在生成能力和图像重建质量上达到了较为稳定的状态。

在进阶任务中，条件变分自编码器（Conditional VAE）通过引入标签条件信息，使得模型不仅能够重建图像，还能够根据给定的标签条件生成特定类别的图像。这一修改提升了模型的灵活性和生成能力，在测试集上也展现了较强的泛化性能。对比实验结果，可以看到随着条件信息的加入，模型生成的图像质量进一步提升，并且能够根据不同标签条件生成多样化的图像，进一步验证了 VAE 在实际应用中的潜力与优势。